

1 Purpose

Our project is a two-game IoT platform built on a pair of PYNQ-Z1 FPGA boards connected via an AWS EC2 relay server, with all game states rendered live in a browser-based interface and projected onto the floor beneath the rover, making the game physically tangible and visually immersive.

Game 1: Maze Runner: The camera (Node A) searches for 4 brightly coloured red squares on the edges of the projected maze and crops and warps its feed to show a perfect top down view of only the maze despite being at an angle. The rover node (Node B) generates a perfect maze in hardware using Kruskal's algorithm, solves it with a BFS hardware accelerator running entirely in programmable logic, and physically navigates the shortest path. Once the first run completes, three blue objects are placed on wall cells of the projected maze; the camera node (Node A) downsamples the image in hardware and then uses HSV thresholding to detect the objects, transmits their grid coordinates to Node B via AWS, and Node B converts those cells from walls to free spaces, re-syncs the updated maze to the server, and runs BFS again, yielding a potentially shorter path if the removed walls open a more direct route.

Game 2: Pac-Man: the rover becomes a user-controlled agent navigating a 7×13 arena via a wireless controller, collecting six coins while evading up to four ghost agents whose pursuit logic follows line-of-sight detection and wall-following and runs entirely in FPGA fabric on Node A, with ghost positions relayed to the server at sub-100 ms round-trip latency. All four ghost instances execute in parallel in programmable logic, each independently computing its next direction every clock cycle, with the full round-trip: from Node A polling the rover's latest position from AWS, feeding it into the FPGA hardware, reading back the four ghost directions over AXI-Lite, and posting the updated positions to the server all completed in under 100 ms, achieving a responsive game experience.

2 System Architecture

The system is organised across three layers: two PYNQ-Z1 FPGA nodes communicating through an AWS EC2 Flask relay server, with all live states rendered in a browser-based interface polled every 300 ms. Throughout both games, AWS acts as the single source of truth, storing maze state, object positions, ghost positions, rover position, and the leaderboard, with all components staying synchronised through it. The architecture diagrams (Figures 1 and 2) distinguish between components running in programmable logic (PL, FPGA fabric) and those running on the processor (PS), a distinction that is central to the design philosophy of this project.

Node B: Rover FPGA.

In Game 1, Node B is the computational core. The Kruskal IP block runs entirely in PL fabric to generate a unique 10×10 maze each run. Before solving, a Hamming(7,4) integrity check runs on the PS to detect and correct any silent bit-flips in the maze data. The BFS IP block then receives the maze over AXI DMA, a single burst transfer with zero CPU involvement, and streams back the shortest path as one-hot direction bytes. The processor translates these into physical stepper motor commands, using gyroscope feedback from the MPU-6500 over a MicroBlaze-managed I2C bus to achieve accurate 90° turns and straight-line correction. The solved maze and rover position are continuously POSTed to AWS for projection. In Game 2, Node B executes moves received from a wireless controller, each validated server-side by AWS before physical execution, with motor and gyroscope control unchanged.

AWS EC2: Flask Relay Server.

The server holds all live state in memory and persists it across six DynamoDB tables: **MazeTable** (all maze layouts ever synced), **RoverLogs** (per-move telemetry), **GhostLogs** (ghost movement events), **VisionPositions** (latest camera snapshot), **Leaderboard** (session outcomes), and **VoiceCommands** (spoken instructions with timestamps). In Game 1 it relays the maze grid and object coordinates between nodes. In Game 2 it validates every rover move against the maze grid, tracks coin collection and ghost collisions, and drives the leaderboard. The web viewer reads `/stream_solver` and `/stream_arena` at 300 ms intervals, redrawing the grid and all agent positions in real time.

Node A: Camera and Ghost FPGA.

In Game 1, Node A runs the full camera pipeline. Raw 640×480 frames are streamed from the camera into PL via AXI VDMA, where a hardware downsampler reduces the resolution to 320×240 and applies a 2×1 averaging filter before passing the frame to the ARM Processor. OpenCV on the PS then performs lens undistortion, HSV colour thresholding to locate four red corner markers, and a perspective warp, averaged over 30 frames, to produce a corrected top-down 500×500 view. Blue object detection then identifies up to three wall-removal candidates, maps their pixel centroids to 10×10 grid coordinates, and POSTs them to AWS. In Game 2, Node A runs four **ghost_top** instances in parallel in PL fabric, each receiving ghost and rover grid coordinates via GPIO and outputting a direction every clock cycle through a 3-stage pipeline (this optimization technique will be discussed further in the sections below). The processor reads back all four directions over AXI-Lite and relays them to AWS via `/update_ghost`.

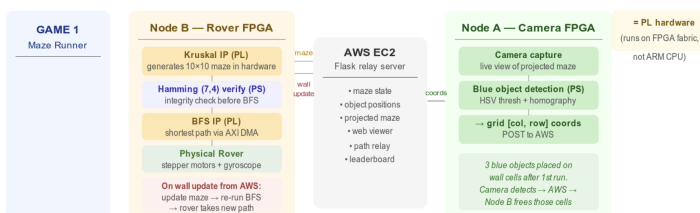


Figure 1: System architecture for Game 1 (Maze Runner)

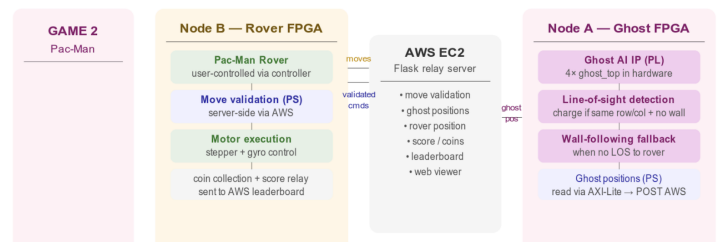


Figure 2: System architecture for Game 2 (Pac-Man)

3 Design Decisions, Evaluation, and Optimizations

3.1 Camera

The camera acts as our real-world sensor for dynamically detecting and processing the environment. Raw frames are ingested through the USB interface; due to USB bandwidth limitations, the 1080p webcam is forced to 640×480. A 2×1 averaging hardware downsampler in PL fabric produces a smoothed 320×240 (240p) output at 9–14 FPS before any software processing. We evaluated a higher-resolution path by routing the camera through a laptop running OBS and outputting via HDMI to the PYNQ board, which yielded 360p, a 50% resolution improvement. However, post-processing frame rates fell to 6–8 FPS on both approaches, revealing that the bottleneck was OpenCV on the ARM CPU, not the capture resolution. Since our detection targets are 5 cm×5 cm brightly coloured squares, reliably detected at 240p, the HDMI path introduced unnecessary hardware complexity (a dedicated laptop) for no tangible benefit in object detection accuracy. We therefore retained the simpler USB pipeline.

3.2 Rover

Precision was critical because the maze is projected on the floor: positional errors make the rover appear to drive into walls even when following the correct path. **Distance control:** NEMA 17 stepper motors were chosen over DC motors for their discrete step behaviour, allowing forward distance to be determined purely by step count from wheel diameter and grid square size, no closed-loop distance correction required. **Turning and drift:** wheel slip caused inaccurate turns and forward drift, so a 3-axis gyroscope MPU-6500 IMU was integrated which communicates with the PYNQ board via I2C. This protocol was chosen as it only requires SDA and SCL wires, leaving sufficient GPIO pins for the stepper motor drivers, and allows for very easy expansion to more sensors if needed. Because Linux’s scheduler cannot guarantee the microsecond timing required for I2C, a MicroBlaze soft-processor was instantiated in PL fabric running bare-metal C, handling all I2C independently of the OS and delivering clean gyroscope data to the Python layer. Turns became gyro-guided: motors spin until the IMU confirms 90°. For straight-line drift, the rover tracks an absolute heading from its starting position, detecting any deviation after each forward step and issuing a micro-correction before the next instruction, preventing errors from compounding over a long path. This yielded a closed-loop directional control system that significantly improved end-point accuracy.

3.3 BFS Algorithm

The BFS core comprises three sub-modules orchestrated by a top-level FSM. `bfs_fsm.sv` implements a 3-state Moore FSM (IDLE → DIST → BT → IDLE), asserting `dist_en` and `bt_en` in mutually exclusive states so the shared `dist_table[]` is written by exactly one module at a time (see Figure 3). `bfs_distmap_10x10.sv` implements BFS as wavefront relaxation: starting from the goal cell at distance 0, it scans all 100 cells repeatedly, assigning distance $d+1$ to any free neighbour of a cell at distance d , until no cell updates in a full pass. `bfs_backtrace_dist.sv` reconstructs the path by walking from the start cell downhill through `dist_table[]`, always stepping to the neighbour at distance cur_d-1 , outputting 4-bit one-hot direction bytes (0001=N, 0010=S, 0100=W, 1000=E), and streaming the result back over AXI DMA.

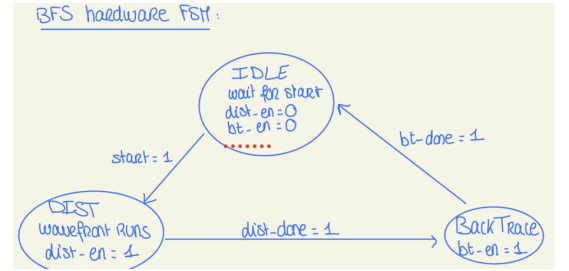


Figure 3: BFS hardware FSM: 3-state Moore machine controlling the distmap and backtrace sub-modules in mutually exclusive phases.

AXI-DMA vs AXI-Lite: AXI-Lite was prototyped first: the 100-cell grid packs into four 32-bit registers, but reading back a 99-step path requires 25 register reads with CPU polling. AXI DMA with AXI-Stream transfers the full maze in a single burst and streams the path back in one receive transaction, with the ARM waiting on an interrupt rather than polling, freeing the CPU entirely during the transfer.

Pipeline optimisation: the initial design failed timing at 100 MHz (WNS = −1.9 ns). Vivado identified the critical path in `bfs_distmap_10x10.sv`: the combinational chain `dist_table[neighbour]` read → compare with $d+1$ → conditional write exceeded the 10 ns clock period. A 3-stage pipeline was introduced: Stage 1 registers the center-cell validity check; Stage 2 reads all four neighbour entries in parallel; Stage 3 performs conditional writes on fully registered values. A one-cycle `S_FLUSH` state drains the pipeline after the final cell. Timing closure is achieved at 100 MHz (WNS = +0.5 ns) at a cost of only 19 additional cycles (≈1% overhead).

3.4 A* Algorithm

The A* accelerator uses the identical AXI-Stream and DMA interface described in Section 5.3, the maze is transferred in a single burst and the path streamed back with `tlast` on the final byte. The FSM extends the BFS design with a fourth state (IDLE → UPDATE → MOVE → BACKTRACING), where UPDATE expands the current node’s four neighbours using the Manhattan distance heuristic $h = |row - end_row| + |col - end_col|$ and inserts valid candidates into a flat 100-entry priority queue. **Sequential neighbour expansion.** In each UPDATE state the four neighbours of the current node must be evaluated and potentially inserted into the priority queue. A parallel implementation would require four simultaneous write ports into the priority queue array, multiplying the insertion logic and routing resources by four. Instead, `update_block` uses a 2-bit `dir_counter` that increments every clock cycle, evaluating one neighbour per cycle and issuing at most one queue insertion per cycle.

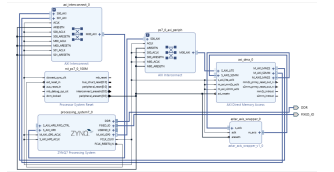


Figure 4: A* Vivado Hardware Block Design

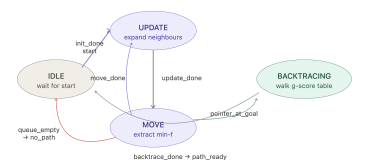


Figure 5: A* Finite State Machine

This reduces the insertion logic to a single write port (eliminating the need for multi-port arbitration and reducing LUT usage and routing complexity). This comes at the cost of four cycles per expansion, which is negligible given the overall path latency.

G-score table as implicit closed set. A separate closed-set array was avoided by initialising all 100 *g*-score entries to 127 and treating any entry still at 127 as unvisited. The `nb_not_visited` check is therefore a single equality comparison, saving both LUTs and a second 100-entry array.

3.5 Path Finding Algorithm Comparison

Table 1: BFS and A* hardware timing and resource comparison.

Design	WNS (ns)	Fmax	LUTs	FFs
BFS, unpipelined	−1.9	≈83 MHz	5521	4205
BFS, pipelined	+0.5	≥100 MHz	5681	4692
A*, unpipelined	−294.5	≈3.3 MHz	8324	6995
A*, pipelined	−10.2	≈49.5 MHz	8469	7391

Note: BFS (pipelined) only used 10.4% of the available PL while A* (pipelined) used 15.9%, showing BFS is more hardware efficient. BFS also had a positive slack while A* had a negative slack of 10.2 ns, showing that BFS could utilize the full clock speed of the FPGA.

3.6 Ghosts

Each ghost instance (`ghost_newmaze_top.sv`) implements a fully pipelined hardware module that produces a new direction every clock cycle. The design is built around three key decisions.

Hardcoded maze as combinational ROM. The 7×13 arena is stored as a 91-bit constant defined as a `localparam`. Both the LOS scan and the wall-check stages read it as combinational logic with zero memory-access latency. A BRAM-loaded dynamic maze would allow runtime reconfiguration but would introduce a memory interface, an initialisation protocol, and additional timing constraints. For a fixed game arena this overhead has no benefit.

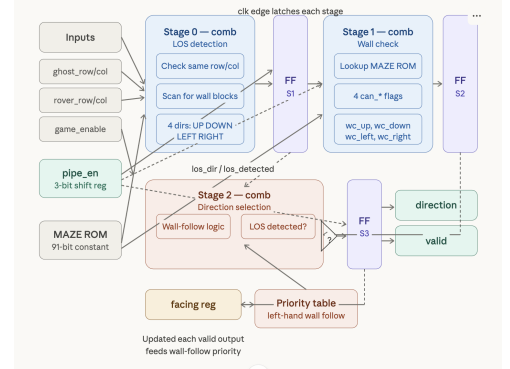


Figure 6: Ghost Hardware 3-stage pipeline

Stage 0 (combinational) performs LOS detection: four independent `always_comb` blocks scan the MAZE constant along the ghost’s row and column in all four directions simultaneously, checking every intervening cell for a wall bit. This parallel scan is the most combinational expensive stage but resolves within one clock.

Optimization for this stage: Parallel LOS detection. The four line-of-sight checks (`los_up_clear`, `los_down_clear`, `los_left_clear`, `los_right_clear`) are implemented as four fully independent `always_comb` blocks, meaning all four direction scans execute simultaneously in the same combinational stage. The latency of Stage 0 is therefore determined by the worst-case single-direction scan rather than the sum of all four. A sequential implementation checking one direction at a time would multiply the combinational delay by four, directly increasing the critical path length and pushing the design below the 100 MHz timing target.

Stage 1 (registered at FF S1) performs the wall check: four combinational flags (`wc_up/down/left/right`) are asserted if the adjacent cell in each direction is free, reading the MAZE constant at the ghost’s registered position.

Stage 2 (registered at FF S2) selects the final direction: if LOS was detected the ghost charges directly; otherwise a priority table indexed by the `facing` register applies a wall-following order. The output is latched at FF S3, asserting `valid` for one cycle.

Four `ghost_top` instances are instantiated in parallel inside `ghost_multi_axi_lite.sv`, sharing the same rover position and `game_enable` inputs. Ghost positions are written via five GPIO blocks (one per ghost + one for the rover). The processor reads back all four directions from AXI-Lite status registers at offsets 0x00–0x0C.

Table 2: Ghost AI hardware timing and resource utilisation.

Design	WNS (ns)	Fmax	LUTs	FFs
Ghost, 4× parallel (pipelined)	+10.96	100 MHz	1839	1178

Note: The 4× parallel ghost pipeline utilizes only 3.4% of the PL, demonstrating high hardware efficiency. The positive slack of +10.96 ns confirms the design comfortably meets the 100 MHz system clock requirement, allowing room for a more complex algorithm.

4 Performance Metrics

To test the various parts of our systems we had to determine different metrics to evaluate the performance of each component. For the purely programmable logic circuits (e.g. the ghosts and the maze solvers/generator) we decided to measure their performance by number of LUTs used and Worst Negative Slack, these metrics were chosen as it determined the percentage of the total PL occupied and the maximum frequency we could run the system at. Tolerance in these would allow the design to be scaled for bigger mazes or more ghosts or adding more complexities in the algorithms for future levels. For the camera we measured the frames per second after software and hardware processing, as it would determine the latency of the system with a higher frame rate allowing for a, marginally, faster response in object detection. For the software and server interactions we measured the end to end latency, from an instruction being sent to the server to a response being received on either FPGA. We aimed to keep this below 100 ms to make

the whole process feel real time to the player.

5 Testing

5.1 Individual Subsystem Testing

Rover: Testing focused on two metrics: distance accuracy and turning consistency. Steps-per-cm was calibrated empirically across multiple runs on the target surface, accounting for wheel slip and friction. Three wheel design iterations were evaluated: (1) tape on the back wheels to enforce sliding, abandoned due to unpredictable behaviour; (2) 3D-printed wheels without tyres rejected due to insufficient grip; (3) 3D-printed wheels with a redesigned axle hole for a tighter fit, which produced significantly more consistent and repeatable movement. Battery voltage was also increased from the recommended 12 V to 18 V, which widened the operating window, previously rover performance degraded sharply below 10 V.

Camera: Frame rate was measured for both the HDMI 360p and USB 240p pipelines using a dedicated FPS counter averaged over an extended period, with the scene held constant as a control variable. Both pipelines averaged 6–8 FPS after hardware and software processing, with the rate depending on scene complexity. Since both inputs achieved higher FPS on raw capture alone, the similarity was attributed to a CPU bottleneck in the OpenCV processing layer rather than the capture interface. No quantifiable frame rate difference was found between the two resolutions despite the higher pixel count of the HDMI input. We also tested the object detection in various lighting environments to determine its limits. We found that we could adjust the parameters in software for the intensity of the colour. However, we would still need, at later times of day or during intense cloud cover, a brighter environment to be able to accurately detect the blue stickers when on top of the green squares. For this we added a "blank maze" mode where we would temporarily clean the maze to make the tiles all white, increasing the contrast between the sticker and the ground, before going back to our walled maze. This led to our sticker detection working during any time of day.

BFS and A*: The BFS hardware was verified first in cycle-accurate simulation using Verilator, which compiles the SystemVerilog RTL to a C++ executable (`sim_main.cpp` instantiates `Vtb_bfs_top`, drives the clock via `Verilated::timeInc(1)`, and runs until `gotFinish()`). The testbench `tb_bfs_top.sv` loads a maze array, pulses `start`, and awaits `path_ready` or `no_path` before printing the maze and full direction sequence. Four test cases were covered: (1) a hand-crafted corridor maze with a known optimal path, manually verified against the expected direction sequence; (2) 20 randomly generated mazes (seeds 0–19), each compared against a software BFS reference to confirm path length equality; (3) an unsolvable maze with the goal cell walled off, confirming `no_path` is asserted and no direction output is produced; (4) a fully open 10×10 grid, confirming the returned path length equals 18 (the minimum Manhattan distance from corner to corner). Simulation results and waveforms are documented in the GitHub repository. We used the same methodology to test A*.

Ghosts: The ghost hardware was verified using a dedicated SystemVerilog testbench (`ghost_top_tb.sv`) covering ten test groups. (1) *Reset behaviour:* after asserting `rst`, `direction` must read UP and `valid` must be low. (2) *Wall follower:* ghost placed at start cell (6,0) with the rover out of line-of-sight; the output direction is checked against a `move_legal` function that queries the hardcoded 91-bit MAZE constant, confirming no wall or out-of-bounds move is produced. (3–6) *Line-of-sight in all four directions:* ghost and rover are placed in the same row or column with a clear corridor between them; the expected direction (UP, DOWN, LEFT, RIGHT) is asserted exactly. (7) *Blocked LOS:* a wall is placed between ghost and rover in the same row, confirming the wall-follower takes over and still produces a legal move. (8) *Boundary cells:* ghost is placed at all four corners and near-boundary free cells; `move_legal` confirms no out-of-bounds direction is ever output.

5.2 Integration Testing

Game 1: Maze Runner

The Game 1 integration test showcases the full pipeline across both nodes and the AWS server in two stages.

Stage 1 Solve: Node B loads `kruskal.bit` onto PL fabric and configures the Kruskal IP via AXI-Lite register writes. The IP completes in 1.3 ms, writing the 10×10 maze to a DMA-allocated output buffer. Before any hardware is invoked, a software BFS runs in Python on the processor to confirm the goal is reachable (`Goal reachable: True`), this acts as a correctness gate, ensuring the hardware is following a correct BFS output. The maze is then POSTed to `/sync_maze`, updating the AWS state, DynamoDB archive, and the live floor projection simultaneously. The BFS bitstream is then loaded, the maze transferred via a single AXI DMA burst, and the path received in under 1 ms (`recvchannel done in 0.000s`). The decoded path length is compared against the software BFS reference to confirm optimality. The rover then executes the path physically, with the absolute-heading gyro loop correcting drift at each step.

Stage 2 Wall Removal: three blue objects are placed on wall cells of the projected maze. Node A detects their centroids via HSV thresholding, maps them to grid coordinates, and POSTs to `/push_positions`, with the full camera-to-server pipeline completing in approximately 2 s.

Node B polls `/get_positions`, zeros those cells in the maze array, POSTs the corrected maze to `/sync_maze` to update the projection, and re-runs BFS. In the recorded test run the path shortened from 26 to 18 steps, confirming that the removed walls

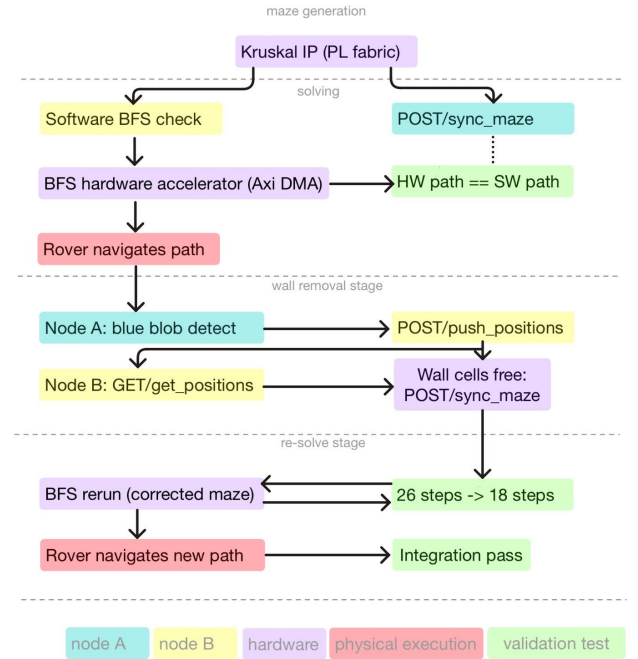


Figure 7: Game 1 Integration Test Flow.

opened a more direct route and that the end-to-end wall-removal pipeline produced a valid updated solution. The rover then navigates the new path to completion.

Game 2: Pac-Man

The Game 2 integration test exercises the real-time two-node communication loop. Node B loads `base.bit`, calibrates the IMU, and on each controller input executes the physical move then POSTs to `/validate_move` for server-side wall, coin, and collision checking. In parallel, Node A loads `ghost_bd.bit`, POSTs the fixed 7×13 arena to `/sync_arena`, then polls `/stream_arena` every 0.6s, writes rover and ghost positions into five GPIO registers, waits 10 ms for the FPGA pipeline to settle, reads four direction outputs via AXI-Lite, and POSTs each to `/update_ghost`. A game ends in either **VICTORY** (all six coins collected) or **KIA** (rover caught by a ghost), with the outcome, agent name, time, and coins written to the DynamoDB leaderboard.

Six scenarios were tested: (1) *coin collection*: rover navigates all six coin positions in sequence, **VICTORY** returned and leaderboard written on the final coin; (2) *wall rejection*: move into a wall cell issued, server returns unchanged position and rover does not move; (3) *ghost-side collision*: rover held stationary until a ghost reaches it via `/update_ghost`, confirming collision detection operates symmetrically from both nodes; (4) *ghost escalation* : ghosts 2–4 confirmed active at 15s intervals via `active_ghosts` in `/stream_arena`

For latency verification, the full Node A ghost loop was timed end-to-end, confirming a round-trip of under 100 ms and web display updates within the 300 ms polling window.

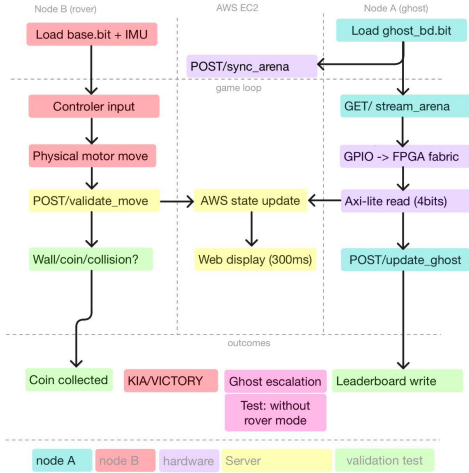


Figure 8: Game 2 Integration Test Flow

6 Extension: Using an FPGA Node to Run Maze Generation and Solving in Parallel

This extension explores the limits of our system in a theoretical setting where the maze changes in real time. Instead of recomputing a single solution after each change, which would introduce latency, we take an AI-inspired approach by solving many possible maze configurations in parallel in hardware using the BFS algorithm.

How did we achieve parallelism? The design achieves parallelism by instantiating multiple independent `maze_worker_engine` modules in hardware within a `generate` loop in `maze_system_top`. Each engine has its own registers, FSMs, and memory. This differs from a `for` loop, which only describes behaviour inside already existing hardware. In FPGA design, each generated instance corresponds to physically separate logic resources such as LUTs, registers, and BRAM. As a result, all engines execute simultaneously rather than time-sharing a processor.

After synthesis, Vivado floor-planning showed that the blue utilised area increased proportionally with the number of solver engines, confirming that the engines are placed as separate computing areas on the chip (Figure 9). Our results were very positive for the 4-engine configuration: we achieved **0.0186 ns** of positive slack.

Performance comparison. We compared performance between solving mazes on the ARM Cortex-A9 versus the FPGA fabric. While the CPU scales sequentially, the FPGA benefits from spatial parallelism and achieves significantly lower execution time as multiple solver engines operate simultaneously (Figures 10 and 11).

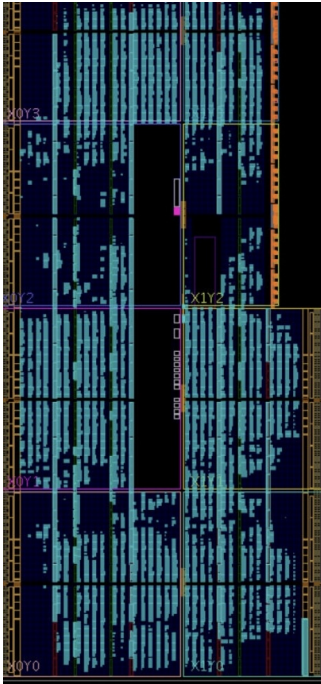


Figure 9: Vivado floorplanning example with 4 solver engines.

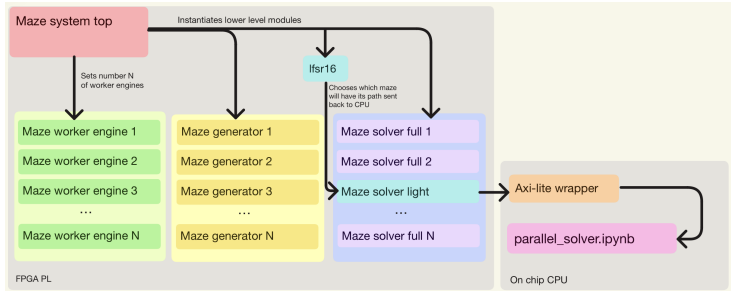


Figure 10: Architecture block diagram.

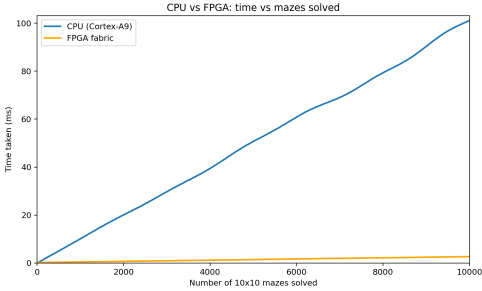


Figure 11: Performance comparison: ARM CPU vs FPGA parallel execution.